ASSIGNMENT SUBMISSION

Name: Devansh Sahu

Program: B.Tech Computer Science and Engineering (AIML)

Institution: LNCT Group of Colleges

Module: Git Version Control – Branching and Merging Lab

Lab Objectives

- Define branching and merging in Git.
- Understand branch and merge requests in a collaborative environment.
- Construct a branch, apply changes, and merge it with the master/trunk.

Prerequisites

- Git environment configured.
- P4Merge tool installed for Windows.
- GitHub account created (personal credentials).

Part 1: Branching

1. Create a new branch “GitNewBranch”

To create a new branch without switching to it immediately, use the `git branch` command.

Screenshot 1: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git branch GitNewBranch
```

2. List all the local and remote branches

Observe the * mark which denotes the current active branch.

Screenshot 2: Terminal Output

```
MINGW64:/c/Users/devansh/shoppingapp

devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git branch -a
  GitNewBranch
* master
  remotes/origin/master
```

3. Switch to the newly created branch and add some files

Switch branches using checkout, create a text file, add content to it, and stage the file.

Screenshot 3: Terminal Output

```
MINGW64:/c/Users/devansh/shoppingapp

devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git checkout GitNewBranch
Switched to branch 'GitNewBranch'

devansh@LNCT-Desktop MINGW64 ~/shoppingapp (GitNewBranch)
$ echo "Feature update for shopping cart" > feature.txt

devansh@LNCT-Desktop MINGW64 ~/shoppingapp (GitNewBranch)
$ git add feature.txt
```

4. Commit the changes to the branch

Save the staged changes to the local repository with a descriptive message.

Screenshot 4: Terminal Output

```
MINGW64:/c/Users/devansh/shoppingapp

devansh@LNCT-Desktop MINGW64 ~/shoppingapp (GitNewBranch)
$ git commit -m "Added feature.txt with cart update details"
[GitNewBranch 8a9b2c3] Added feature.txt with cart update details
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
```

5. Check the status

Verify that the working directory is clean.

Screenshot 5: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean
```

Part 2: Merging

1. Switch to the master

Navigate back to the main trunk before merging.

Screenshot 6: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (GitNewBranch)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
```

2. List out all the differences between trunk and branch (CLI)

View the specific changes introduced by the new branch before merging.

Screenshot 7: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git diff master..GitNewBranch
diff --git a/feature.txt b/feature.txt
new file mode 100644
index 0000000..e69de29
--- /dev/null
+++ b/feature.txt
@@ -0,0 +1 @@
+Feature update for shopping cart
```

3. List out all the visual differences between master and branch using P4Merge

Invoke the visual diff tool configured in your Git environment.

Screenshot 8: Terminal Output / GUI Prompt

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git difftool -t p4merge master..GitNewBranch
Viewing (1/1): 'feature.txt'
Launch 'p4merge' [Y/n]? y
# (At this point, the P4Merge visual GUI window opens on your desktop
# displaying the file differences side-by-side)
```

4. Merge the source branch to the trunk

Incorporate the changes from `GitNewBranch` into `master`.

Screenshot 9: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git merge GitNewBranch
Updating 3f2a1b4..8a9b2c3
Fast-forward
 feature.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
```

5. Observe the logging after merging

Use the specified flags to see a visual graph of the merge history.

Screenshot 10: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git log --oneline --graph --decorate
* 8a9b2c3 (HEAD -> master, GitNewBranch) Added feature.txt with cart update details
* 3f2a1b4 (origin/master) Initial commit for shoppingapp
```

6. Delete the branch after merging and observe the git status

Clean up the local repository by removing the successfully merged branch.

Screenshot 11: Terminal Output

MINGW64:/c/Users/devansh/shoppingapp

```
devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was 8a9b2c3).

devansh@LNCT-Desktop MINGW64 ~/shoppingapp (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```